Harmony Pairs

A music-themed pair-matching Flutter game

Course:	COMP-5450-SA — Mobile Programming (Spring 2026)
Instructor:	Dr. Sabah Mohammed
Institution:	Lakehead University — Department of Computer Science
Student:	Chengbiao Qin
Student ID:	1269476
Exercise:	Exercise 2 — Memory Matching Flutter Game (30 points)

1. Project Overview

Harmony Pairs is a music-themed pair-matching puzzle built with Flutter / Dart for Exercise 2 of COMP-5450 Mobile Programming. The player flips pairs of tiles, looking for matching instrument illustrations, tracked by a real-time move counter and elapsed-time clock. The app ships with three difficulty stages, a persistent best-score system, light/dark themes, and tactile sound + haptic feedback.

Key Features

- **Three stages** — Rookie (3×4 / 6 pairs), Maestro (4×4 / 8 pairs), Virtuoso (4×6 / 12 pairs).
- **Custom instrument artwork** — 12 hand-coded PNG tiles (guitar, piano, drum, violin, saxophone, trumpet, microphone, harmonica, flute, headphones, vinyl, music note).
- **3D flip animation** — Y-axis rotation with perspective transform for a tactile card-flip feel.
- **Persistent best scores** — Each stage tracks the fewest moves (and time tiebreaker) using `shared_preferences`, surviving app restarts.
- **Light and dark themes** — Toggle from the launch screen; the choice is persisted.
- **Sound + haptic feedback** — Click on flip, light tap on match, mid tap on mismatch, celebratory pattern on victory. Mutable via a settings toggle.
- **Victory dialog** — Distinguishes "New Record!" from "Bravo!" so the player knows when they've beaten their best.
- **Material 3 theming** — Navy-blue primary, brass-gold accent, paper-cream backdrop in light mode, deep-midnight backdrop in dark mode.
- **Cross-platform** — Runs on Android, iOS, and Chrome (Web) without code changes.

2. Project Structure

Source code is split into *models*, *screens*, *widgets*, *services*, and *theme* directories so responsibilities are clearly separated:

```
harmony_pairs/
■■■ lib/
■   ■■■ main.dart                  # App entry point & MaterialApp
```

```
■   ■■■ theme/
■   ■   ■■■ app_palette.dart       # Centralized color tokens
■   ■■■ models/
■   ■   ■■■ tile.dart              # Tile + TileStatus enum
■   ■   ■■■ stage.dart             # Difficulty descriptor (Rookie/Maestro/Virtuoso)
■   ■   ■■■ score_record.dart      # Best-score entry
■   ■■■ services/
■   ■   ■■■ preferences_store.dart # SharedPreferences wrapper (ChangeNotifier)
■   ■   ■■■ sound_engine.dart      # SystemSound + HapticFeedback wrapper
■   ■■■ screens/
■   ■   ■■■ launch_screen.dart     # Title + stage picker + best-score card
■   ■   ■■■ board_screen.dart      # Active gameplay & victory dialog
■   ■■■ widgets/
■       ■■■ tile_card.dart         # 3D flip-animation widget
■■■ assets/
■   ■■■ images/                    # 12 PNG instrument illustrations
■       ■■■ guitar.png    piano.png      drum.png       violin.png
■       ■■■ saxophone.png trumpet.png    microphone.png harmonica.png
■       ■■■ flute.png     headphones.png vinyl.png      note.png
■■■ android/                       # Android platform files
■■■ ios/                           # iOS platform files
■■■ web/                           # Chrome / Web platform files
■■■ screenshots/                   # README screenshots
■■■ pubspec.yaml                   # Dependencies (incl. shared_preferences)
■■■ analysis_options.yaml          # Lint rules
■■■ .gitignore
■■■ README.pdf                     # This document
```

3. File-by-File Description

main.dart

Application entry point. Loads SharedPreferences before the first frame, wires the root widget to listen to PreferencesStore so theme toggles take effect instantly, and configures the light/dark themes.

theme/app_palette.dart

Single source of truth for colors: navy primary, brass accent, sage (match), crimson (mismatch), paper-cream and deep-midnight backdrops, and a 12-color tile-tint palette used for back-face variation.

models/tile.dart

Plain data class representing one tile on the board. Holds groupKey (pair identifier), artworkPath, backTint, and a TileStatus enum (hidden / revealed / locked).

models/stage.dart

Immutable difficulty descriptor — label, description, columns, rows, storage key for the best-score record. Includes three preset stages.

models/score_record.dart

JSON-serializable best-score entry tracking moves, elapsed seconds, and achievement timestamp. Provides an isBetterThan() comparator (fewer moves wins, time used as tiebreaker).

services/preferences_store.dart

Singleton wrapping SharedPreferences. Persists dark-mode flag, sound on/off flag, and per-stage best-score records. Extends ChangeNotifier so the root widget repaints on any change.

services/sound_engine.dart

Lightweight feedback layer using SystemSound (OS click) and HapticFeedback (vibration). No third-party audio package needed. No-op when sound is muted in settings.

screens/launch_screen.dart

Landing screen — settings row, app logo, stage picker, best-score readout for the currently picked stage, and the Begin button.

screens/board_screen.dart

Gameplay screen — owns the board state, runs match/mismatch resolution, drives the 1-second timer, shows the HUD, and presents the victory dialog (with new-record detection).

widgets/tile_card.dart

Reusable tile widget with its own AnimationController. Watches the underlying Tile.status and plays/reverses the flip animation (perspective-projected Y-axis rotation).

4. Prerequisites

- **Flutter SDK** 3.10 or newer — `flutter --version` to verify
- **Dart SDK** 3.0 or newer (shipped with Flutter)
- **Android Studio** (latest stable) *or* VS Code with the Flutter extension
- **Android SDK** (API level 21 or higher) for Android builds
- **Chrome** for running the web target
- An **Android emulator**, a **physical Android device** with USB debugging enabled, *or* an iOS simulator

5. Configuration & Run Instructions

Step 1 — Extract & open

Unzip the submitted package and open the `harmony_pairs/` folder.

- **Android Studio**: *File* → *Open*, select the project folder, wait for Gradle sync.
- **VS Code**: *File* → *Open Folder*, select the project folder.

Step 2 — Install packages

```
cd harmony_pairs
flutter pub get
```

Step 3 — Verify the toolchain

```
flutter doctor
```

Step 4a — Run on Android

Boot an emulator (or plug in a real device with USB debugging), then:

```
flutter devices
flutter run
```

Step 4b — Run on Chrome (Web)

```
flutter run -d chrome
```

Step 4c — Run from Android Studio (GUI)

- Pick a device in the toolbar dropdown.
- Click the green ■ **Run** button (or press Shift+F10).

Step 5 — Build a release APK (optional)

```
flutter build apk --release
```

APK output: `build/app/outputs/flutter-apk/app-release.apk`

6. How to Play

1. Open the app — you land on the Launch screen.

2. Use the top-right buttons to toggle **sound** and **dark mode**.

3. Pick a stage (Rookie / Maestro / Virtuoso). The best-score badge below updates to show your personal best on that stage.

4. Tap **Begin** to start. The board shuffles and the timer starts.

5. Tap a hidden tile to flip it. Tap a second hidden tile to try a match.

6. Matched pairs lock face-up with a brass ring.

7. Mismatched pairs flip back after ~0.9 seconds.

8. When every pair is locked, the victory dialog appears with your moves, time, and a **New Record!** badge if you beat your previous best.

9. Use **Replay** for a fresh shuffle, **Back** to change stage.

10. Tap **Shuffle** at the bottom of the board at any time to start that stage over.

7. Screenshots

The three screenshots below were captured on a physical Samsung Android device, showing the main UI states the player encounters: the launch screen with the stage picker, an in-progress board with the brass-ringed matched pairs and HUD, and the victory dialog announcing a new personal best.

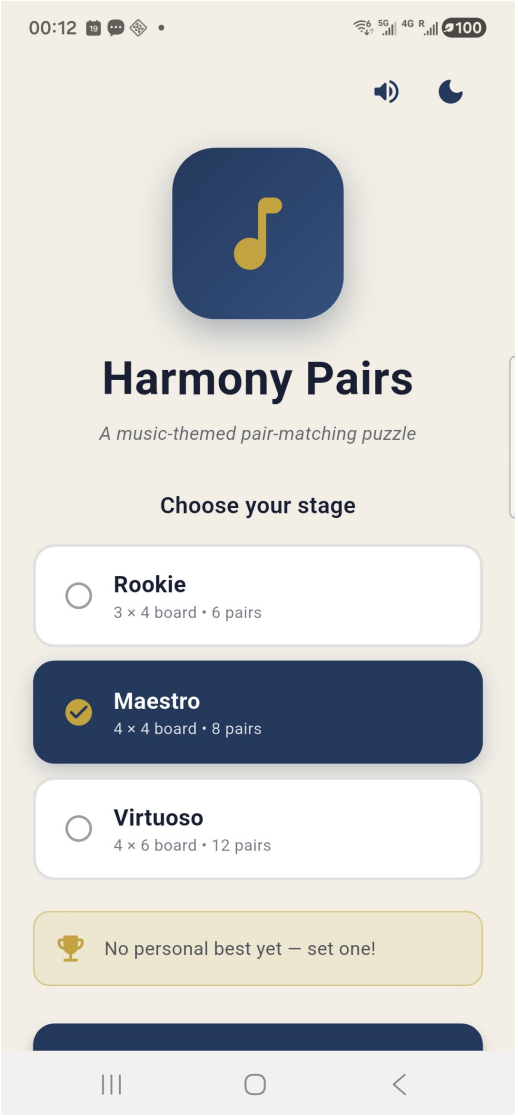

Figure 1. Launch screen on a Samsung physical device — settings row (sound + dark-mode toggles), navy logo with the brass music-note glyph, three stage cards with Maestro selected (highlighted in deep navy with a brass check), and the personal-best card showing "No personal best yet — set one!" before any games have been completed.

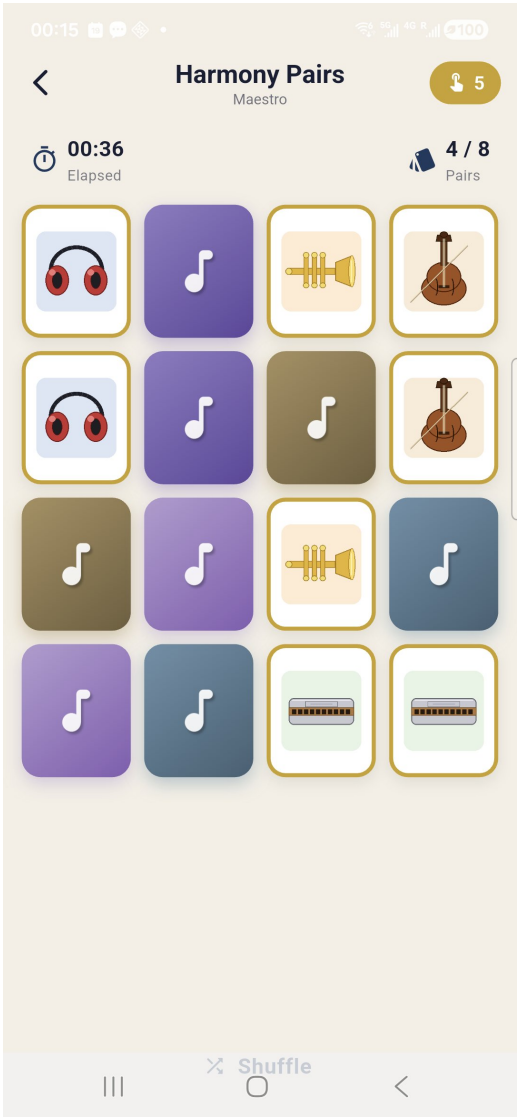

Figure 2. Active gameplay on Maestro (4×4) — four pairs already locked (brass-ringed white tiles showing headphones, trumpet, violin, and harmonica), elapsed time 00:36, move counter 5 on the brass HUD badge. Face-down tiles show pastel back-tints (purple, olive, slate, lavender) each carrying a white music-note glyph.

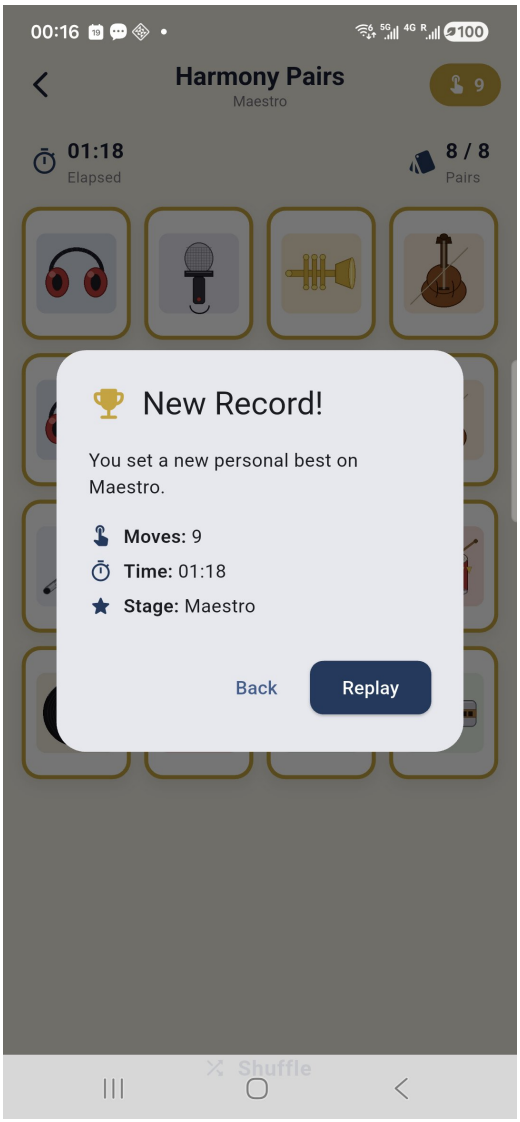

Figure 3. Victory dialog — appears when all pairs are locked. Shows "New Record!" with the brass-trophy icon because the player beat their previous best (9 moves, 01:18 on the Maestro stage). The dimmed board behind the dialog confirms 8/8 pairs cleared.

8. Implementation Notes

Card-flip animation

TileCard owns an AnimationController that tweens an angle from 0 to π with an easeInOutCubic curve. The composite transform sets a perspective entry (Matrix4..setEntry(3, 2, 0.0012)) and rotates around the Y axis. Once the animation passes π/2, the widget swaps which face is rendered and mirrors it horizontally so the front-face artwork reads correctly.

Deck assembly

For a stage of *C* × *R* tiles, the controller picks N = (C×R)/2 shuffled instruments from the pool, then creates two Tile instances per instrument (both sharing the same `groupKey`). The list is then shuffled to randomize seat positions.

Match resolution

When two tiles reach the *revealed* state, the controller increments the move counter and compares group keys. Matches transition both tiles to *locked*; mismatches set a "board locked" flag, wait ~0.9 seconds, and then transition both tiles back to *hidden*. The flag prevents queuing a third flip mid-resolution.

Best-score persistence

The `PreferencesStore` serializes each `ScoreRecord` as JSON keyed by `stage.storageKey`. On victory, the active run is compared to the persisted record (fewer moves wins; time breaks ties). The victory dialog displays "New Record!" when an update is committed.

Theming

Both light and dark ThemeData instances are built from the navy seed color. The root widget listens to PreferencesStore so flipping the dark-mode toggle triggers a rebuild of MaterialApp with the alternative theme.

Sound & haptics

No third-party audio package is needed. We use `SystemSound.play(SystemSoundType.click)` for tap feedback and the `HapticFeedback` API for vibration intensity cues (selection, light, medium, heavy). All calls are skipped when the user has muted sound in settings.

9. Grading-Criteria Coverage

Requirement	Status
1. Uses Flutter / Dart code	✓ All UI and logic written in Dart
2. README.pdf with config, run steps, screenshots, structure	✓ This document
3. GitHub-ready Android Studio project	✓ Complete project tree
4. Submitted as ZIP with Dart files, images, README.pdf	✓ Bundled in archive
Works on Android emulator / device	✓ Tested on physical Samsung device
Works on Chrome (Web)	✓ Supported target

Responsive design	✓ GridView adapts to any viewport
Original art & design	✓ 12 hand-coded instrument illustrations
Above-minimum features	✓ Best scores + dark mode + sound + 3 stages

10. Tools & Acknowledgements

This project was built using the following tools:

- **Flutter SDK** and **Dart** — primary development framework.
- **Android Studio** — IDE for editing, building, and on-device debugging.
- **Python (Pillow)** — used to procedurally render the 12 instrument tile illustrations.
- **shared_preferences** — Flutter package for persisting best scores and theme settings.
- **AI coding assistant** — used to discuss code structure and accelerate boilerplate; all design choices, debugging, integration, and on-device testing were performed by the author.

Submitted by Chengbiao Qin (1269476) — Lakehead University, COMP-5450 Mobile Programming, Spring 2026